# PART 3: COMBINATIONAL LOGIC

## ADDERS & SUBTRACTORS

SAAD BAIG – DIGITAL LOGIC DESIGN


The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from https://www.hasanbaig.com/resources

# BINARY ADDER

| | |
|---|---|
| $0 + 0 = 0$ | Sum of 0 with a carry of 0 |
| $0 + 1 = 1$ | Sum of 1 with a carry of 0 |
| $1 + 0 = 1$ | Sum of 1 with a carry of 0 |
| $1 + 1 = 10$ | Sum of 0 with a carry of 1 |

- When the augend and addend numbers contain more significant digits, the carry obtained from the addition of two bits is added to the next higher order pair of significant bits.
- A combinational circuit that performs the addition of two bits is a half adder. A circuit that performs the addition of three bits (two significant bits and a previous carry) is a full adder.

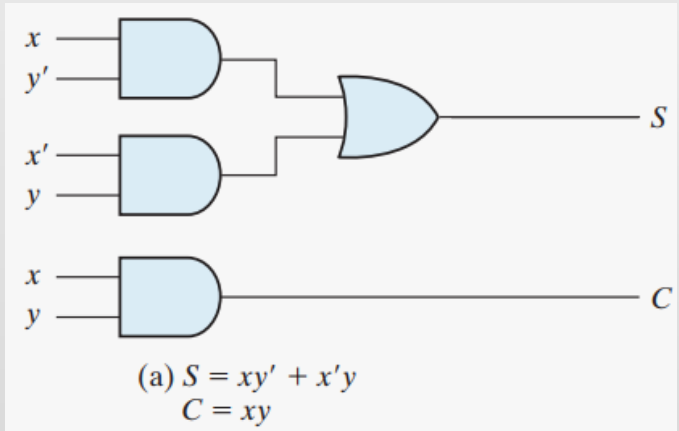
# HALF ADDER

- Number of inputs = 2. Number of outputs = 2 → S = Sum, C = Carry.

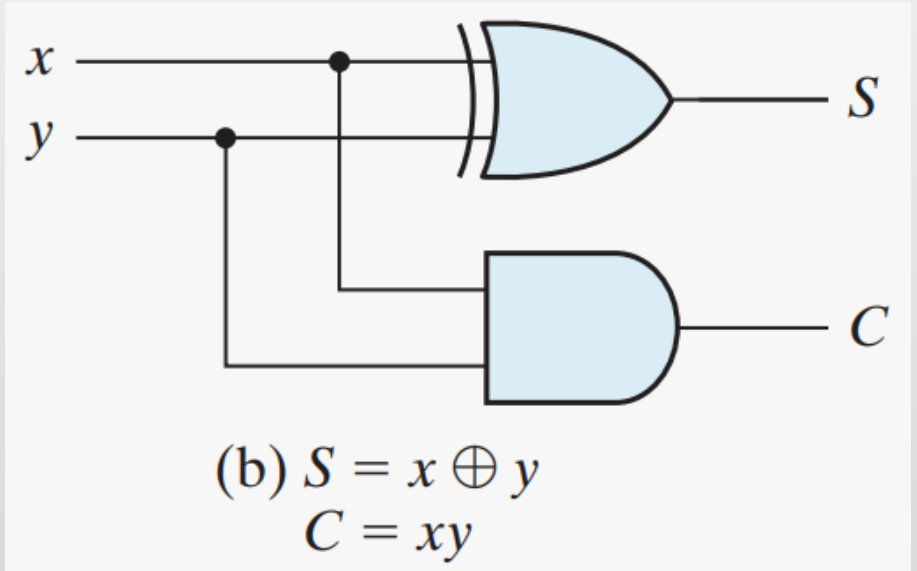

$$S = x'y + xy'$$
$$C = xy$$

**Half Adder**

| x | y | C | S |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |




(a) $S = xy' + x'y$
$C = xy$

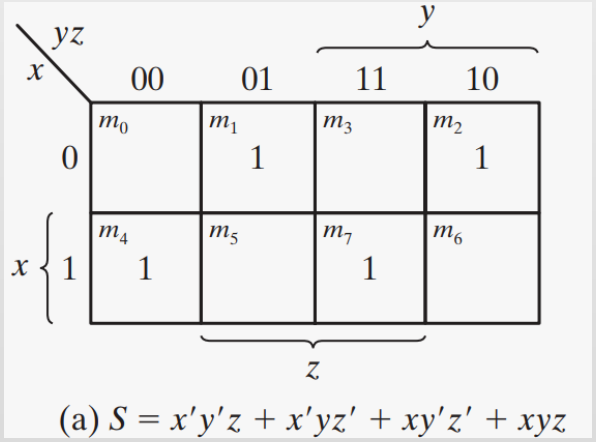



(b) $S = x \oplus y$
$C = xy$

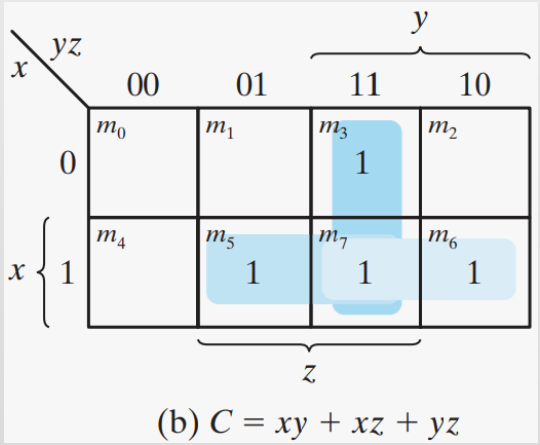
# FULL ADDER

- Number of inputs = 3. Number of outputs = 2 → S = Sum, C = Carry.


$$S = x'y'z + x'yz' + xy'z' + xyz$$
$$C = xy + xz + yz$$




(a) $S = x'y'z + x'yz' + xy'z' + xyz$




(b) $C = xy + xz + yz$

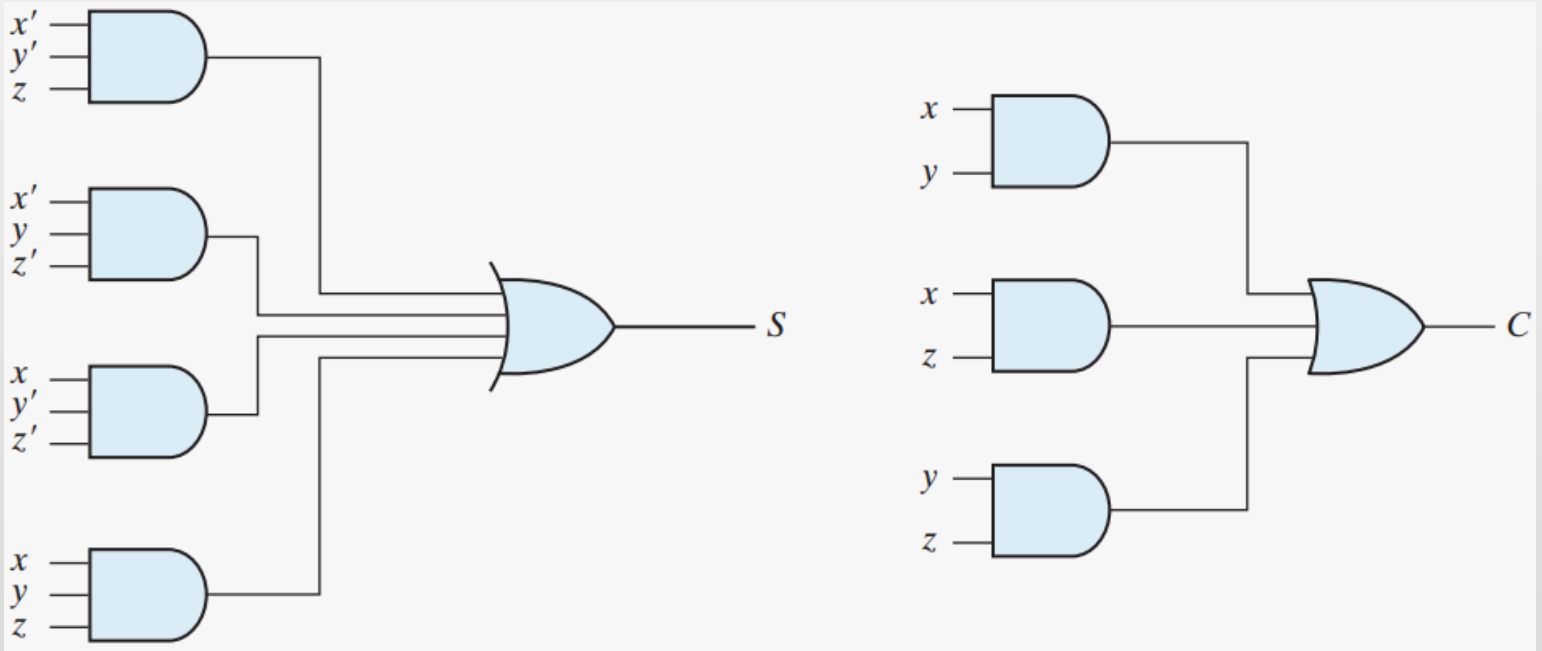
**Full Adder**

| x | y | z | C | S |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

# FULL ADDER

- Full adder implementation in SOP form.

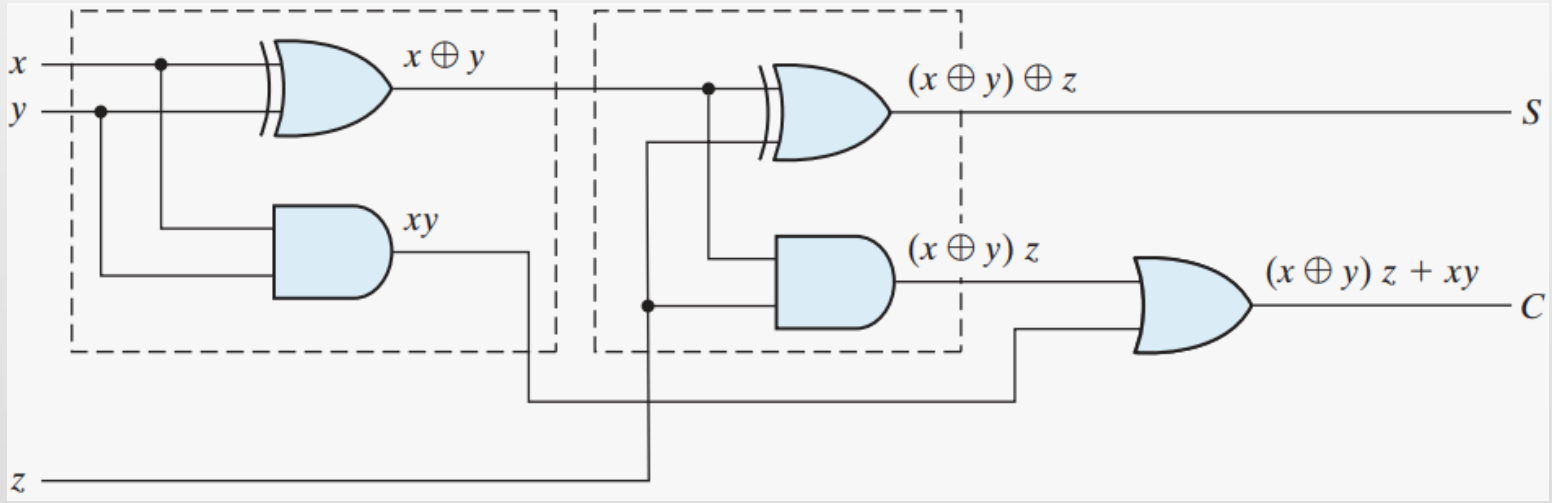

$$S = x'y'z + x'yz' + xy'z' + xyz$$
$$C = xy + xz + yz$$



# FULL ADDER

- Full adder with two half adders and an OR gate.

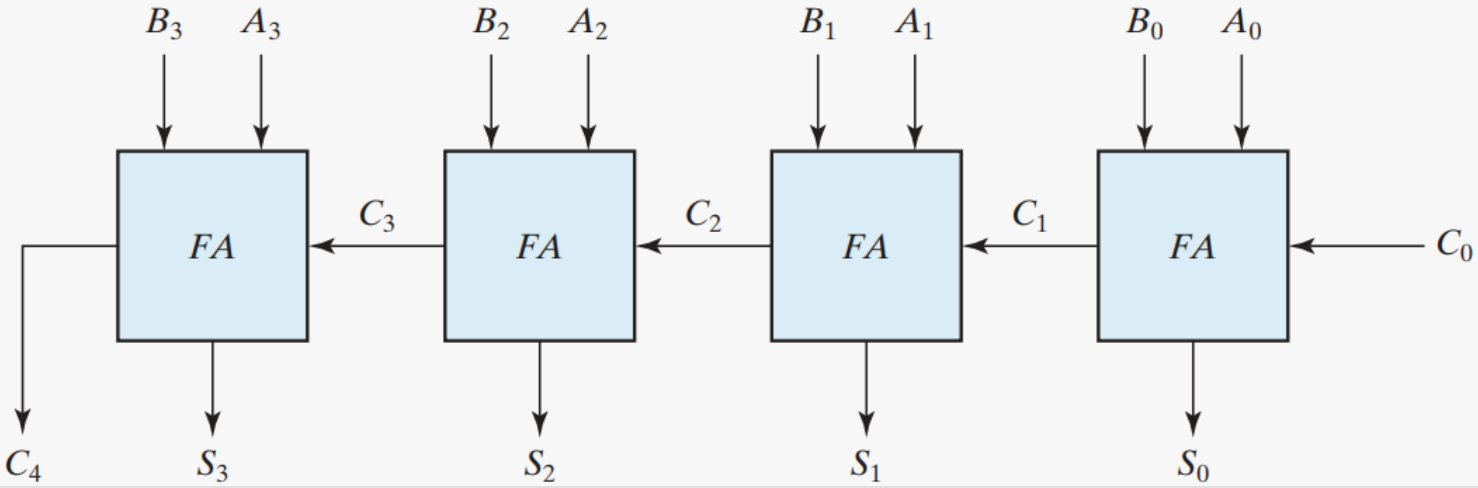

$$S = x'y'z + x'yz' + xy'z' + xyz$$
$$C = xy + xz + yz$$



# BINARY ADDER

- A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers.
- It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain.
- Addition of n-bit numbers requires a chain of n full adders or a chain of one-half adder and n -1 full adders. In the former case, the input carry to the least significant position is fixed at 0.

# 4–BIT BINARY ADDER

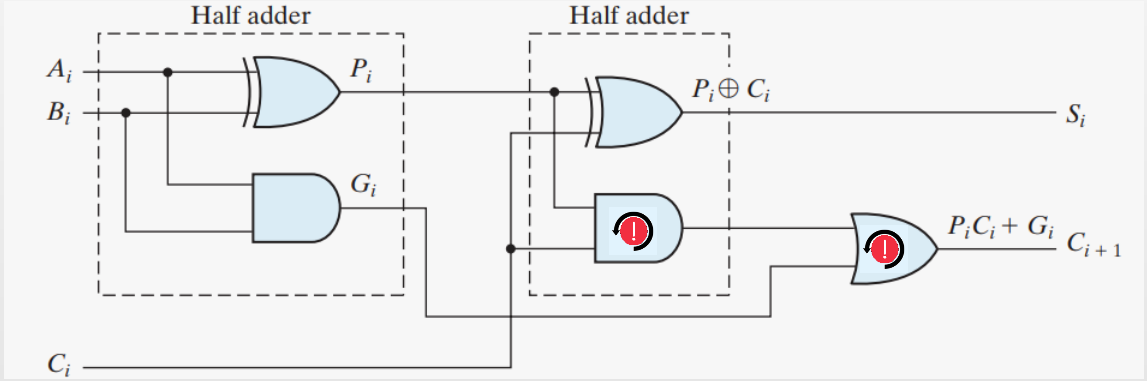


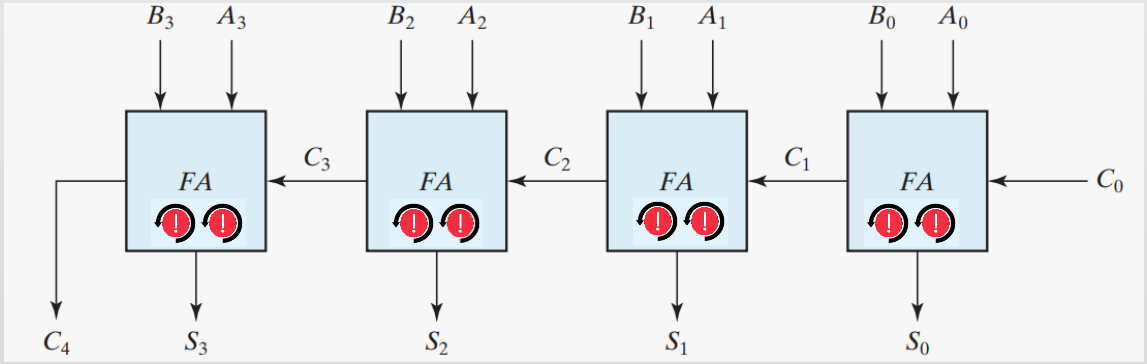

- Example: Addition of 1011 with 0011

| **Subscript *i*:** | **3** | **2** | **1** | **0** | |
|---|---|---|---|---|---|
| Input carry | 0 | 1 | 1 | 0 | $C_i$ |
| Augend | 1 | 0 | 1 | 1 | $A_i$ |
| Addend | 0 | 0 | 1 | 1 | $B_i$ |
| Sum | 1 | 1 | 1 | 0 | $S_i$ |
| Output carry | 0 | 0 | 1 | 1 | $C_{i+1}$ |

# CARRY PROPAGATION

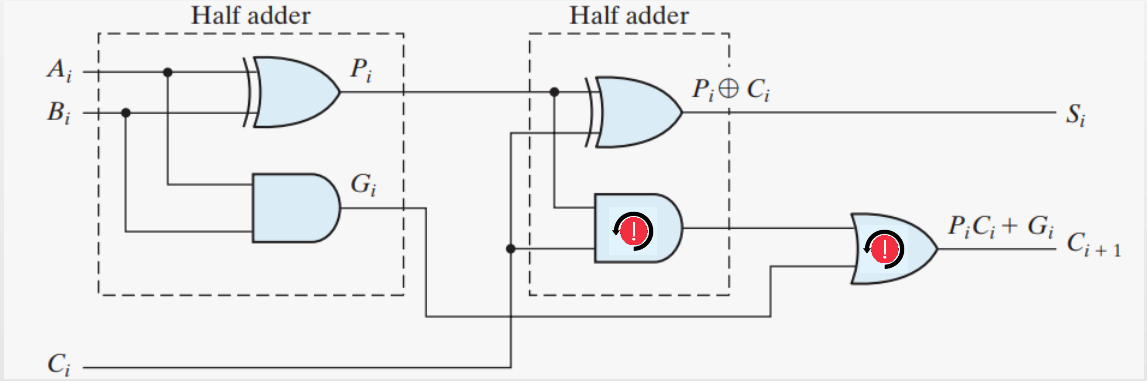
- The addition of two binary numbers in parallel implies that all the bits of the augend and addend are available for computation at the same time.
- The signal from the input carry $C_i$ to the output carry $C_{i+1}$ propagates through an AND gate and an OR gate, which constitute two gate levels.
- For *n*-bit adder, there are 2*n* gate level delays for carry to propagate from input to output.

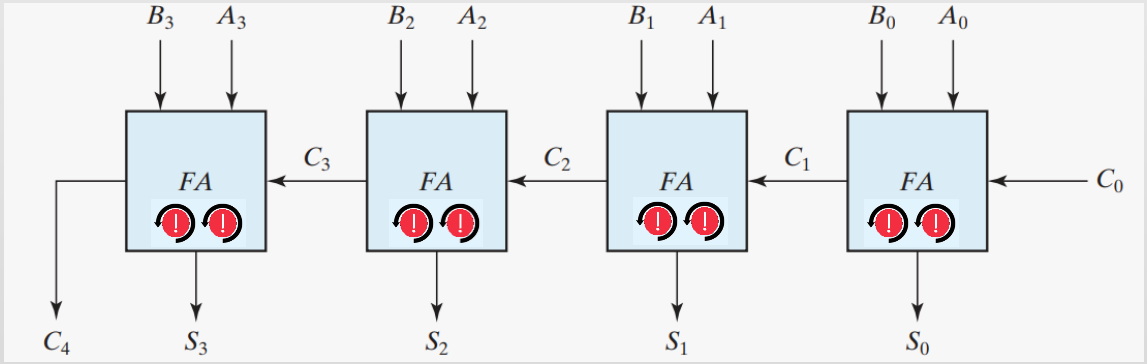


# CARRY PROPAGATION

- The carry propagation time is an important attribute of the adder because it limits the speed with which two numbers are added.
- A solution is to increase the complexity of the equipment in such a way that the carry delay time is reduced.
- There are several techniques for reducing the carry propagation time in a parallel adder. The most widely used technique employs the principle of carry look-ahead logic.

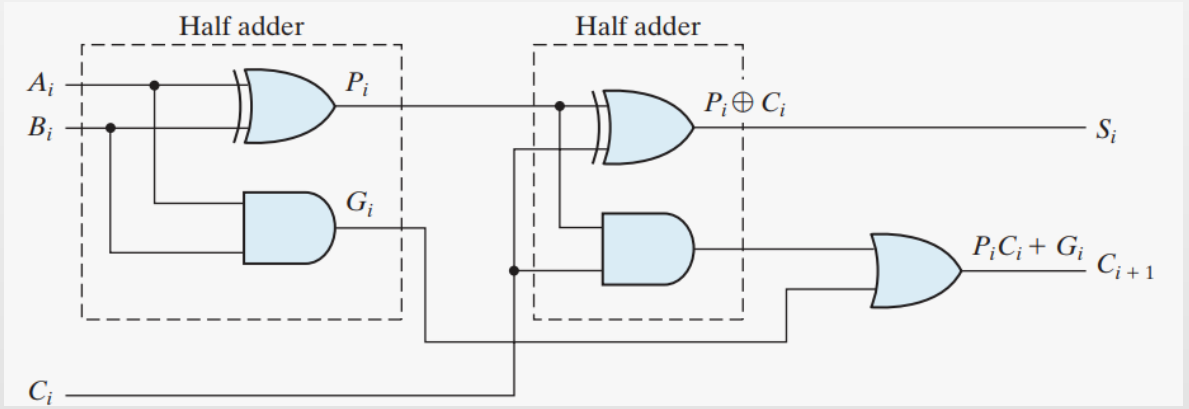


# CARRY LOOK-AHEAD LOGIC

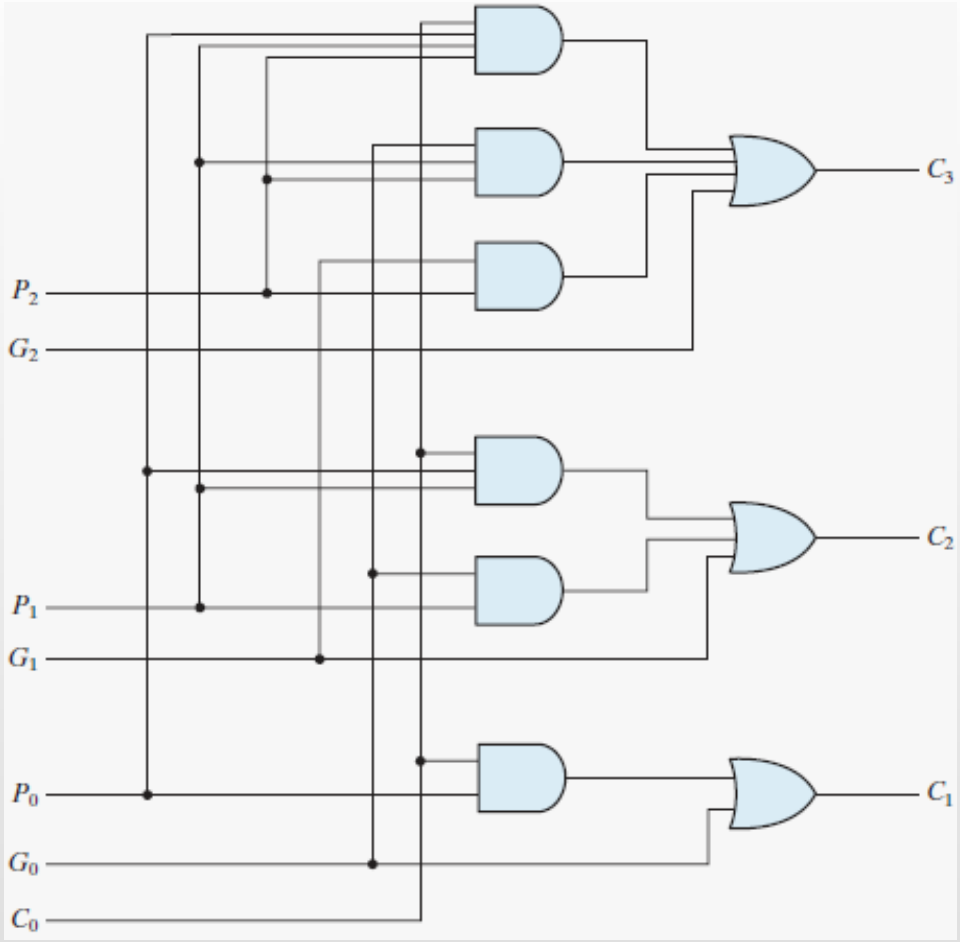
1. Consider the circuit of the full adder shown.

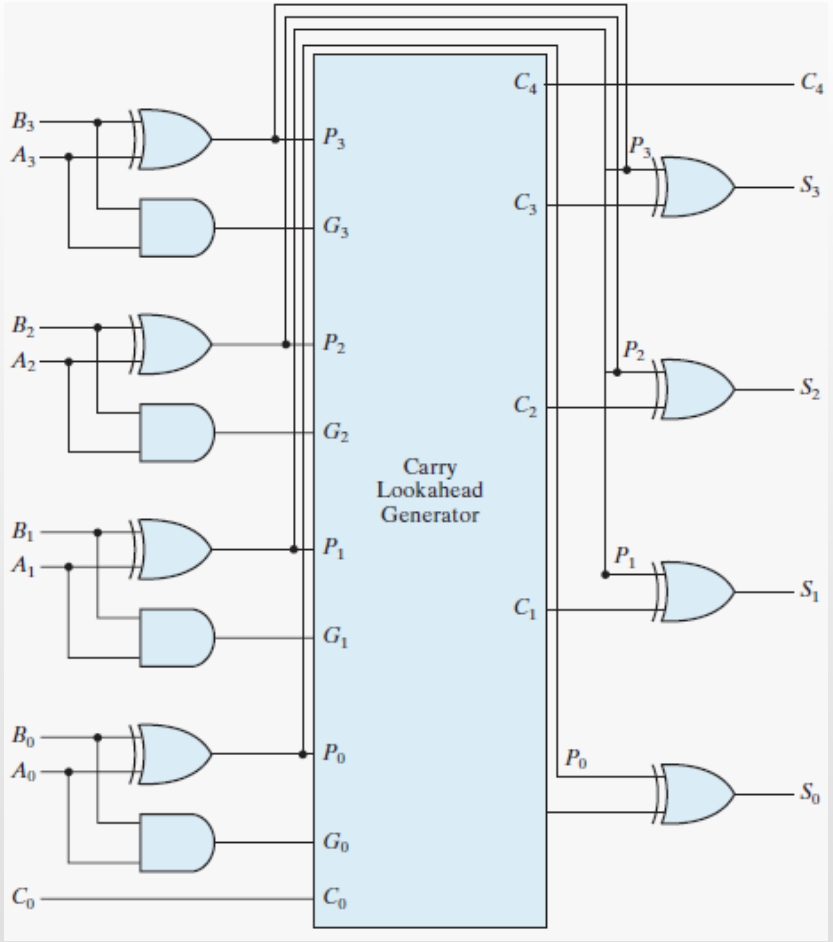
$$P_i = A_i \oplus B_i \qquad S_i = P_i \oplus C_i$$
$$G_i = A_i B_i \qquad C_{i+1} = G_i + P_i C_i$$




2. We now write the Boolean functions for the carry outputs of each stage and substitute the value of each Ci from the previous equations.

$$C_0 = \text{input carry} \qquad C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$
$$C_1 = G_0 + P_0 C_0 \qquad C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

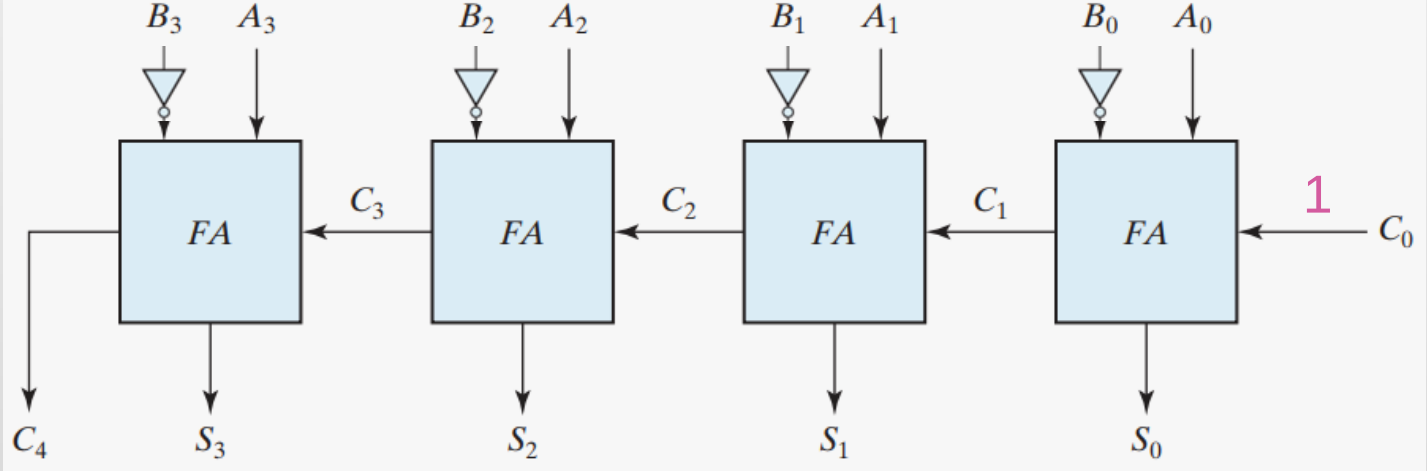



# 4–BIT ADDER WITH CARRY LOOK–AHEAD GENERATOR

# BINARY SUBTRACTOR

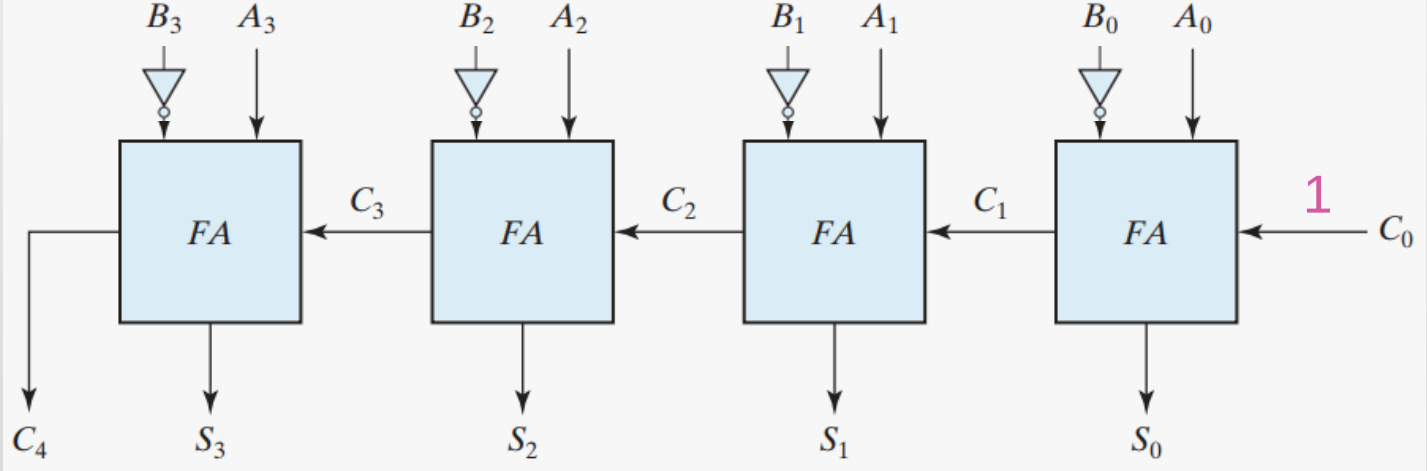
- Subtraction of unsigned binary numbers can be done by means of complements.
- Subtraction A – B can be done by taking the 2's complement of B and adding it to A. The 2's complement can be implemented using inverters (1's complement), and 1 can be added to the sum through the input carry.



# BINARY SUBTRACTOR

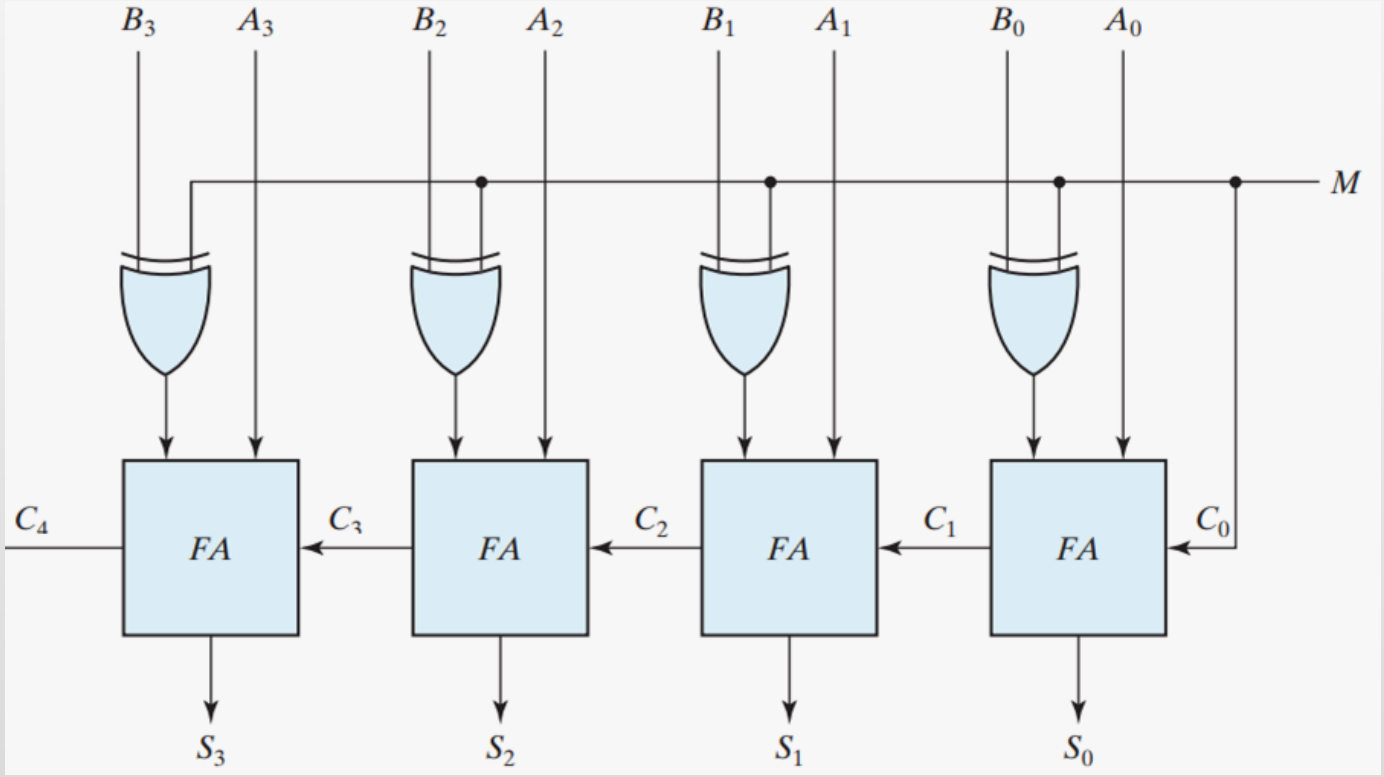
- For unsigned numbers A and B, operation (A – B) yields:
  - If A ≥ B, then $S_3S_2S_1S_0$ = (A – B)
  - If A < B, then $S_3S_2S_1S_0$ = 2's complement of (B – A)

- For signed numbers, the result is (A – B) provided there is no overflow.



# BINARY ADDER–SUBTRACTOR

- The addition and subtraction operations can be combined into one circuit with one common binary adder by including an exclusive-OR gate with each full adder.
- A mode bit M determines whether the operation is addition or subtraction.
  - B $\oplus$ 0 = B,  B $\oplus$ 1 = B'



# OVERFLOW IN BINARY ADDER–SUBTRACTOR


- When two numbers with *n* digits each are added and the sum is a number occupying *n* + *1* digits, we say that an overflow occurred.
- For unsigned numbers: An overflow is detected from the end carry out of the most significant position. (i.e. $C_4$ in 4-bit adder-subtractor).
- For signed numbers:
  1. The end carry DOES NOT indicate sign.
  2. Overflow CANNOT occur after the addition of one positive number with one negative number.
  3. An overflow MAY occur if the two numbers added are both positive or both negative.

# OVERFLOW IN BINARY ADDER–SUBTRACTOR

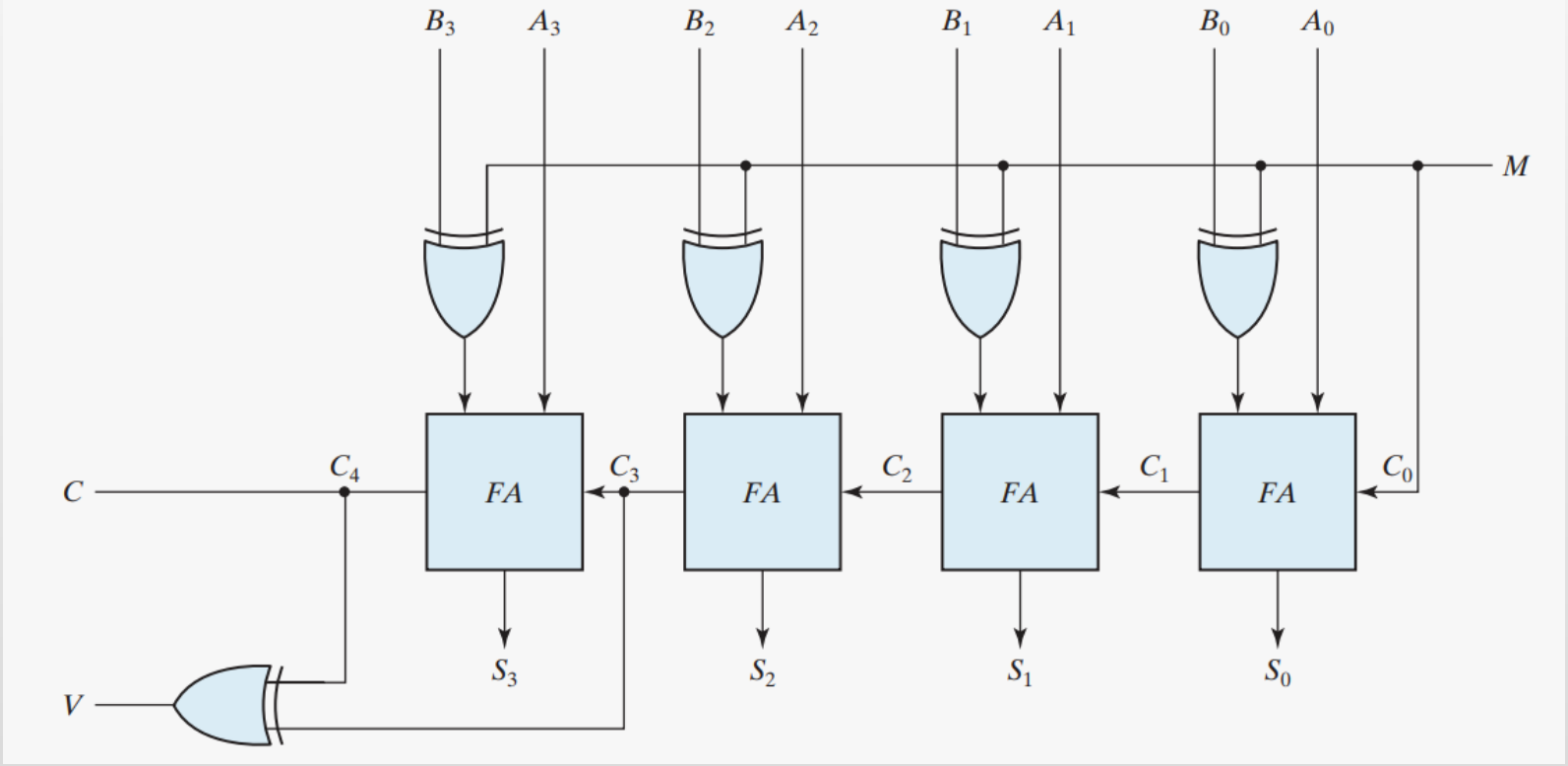
- Example: Arithmetic of 2 signed numbers.

| | | |
|---|---|---|
| | 0 1 | |
| +70 | 0 1000110 | −70 |
| +80 | 0 1010000 | −80 |
| +150 | 1 0010110 | −150 |
| | 1 0 | |
| | 1 0111010 | |
| | 1 0110000 | |
| | 0 1101010 | |

- An overflow condition can be detected by observing the carry into the sign bit position and the carry out of the sign bit position. If these two carries are not equal, an overflow has occurred.

# OVERFLOW IN BINARY ADDER–SUBTRACTOR

- An overflow condition can be detected by observing the carry into the sign bit position and the carry out of the sign bit position.
- If these two carries are not equal, an overflow has occurred.



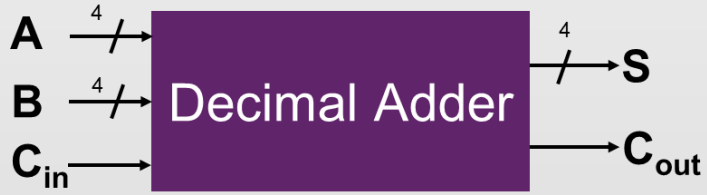
# OVERFLOW IN BINARY ADDER–SUBTRACTOR

- Summary:

1. If the two binary numbers are considered to be unsigned, then the *C* bit detects a carry after addition or a borrow after subtraction.
2. If the numbers are considered to be signed, then the *V* bit detects an overflow.
3. If V = 0 after an addition or subtraction, then no overflow occurred and the *n*-bit result is correct.
4. If V = 1, then the result of the operation contains $n + 1$ bits, but only the rightmost n bits of the number fit in the space available, so an overflow has occurred. The $(n + 1)^{th}$ bit is the actual sign and has been shifted out of position.

# DECIMAL ADDER

- Computers or calculators that perform arithmetic operations directly in the decimal number system represent decimal numbers in binary coded form.
- A decimal adder requires a minimum of nine inputs and five outputs.
- For BCD adder the sum cannot exceed 19
  - Example: 9+9+1 = 19
- Design approach:
  1. Utilize 4-bit adder to add two digits and a carry then covert the result into BCD
  2. For n-bit decimal adder cascade *n* BCD adder stages



# DECIMAL ADDER

*No conversion needed here*

*Derivation of BCD Adder*

| Binary Sum | | | | | BCD Sum | | | | | Decimal |
|---|---|---|---|---|---|---|---|---|---|---|
| K | $Z_8$ | $Z_4$ | $Z_2$ | $Z_1$ | C | $S_8$ | $S_4$ | $S_2$ | $S_1$ | |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 2 |
| 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 3 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 4 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 5 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 6 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 7 |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 8 |
| 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 9 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 10 |
| 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 11 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 12 |
| 0 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 13 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 14 |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 15 |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 16 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 17 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 18 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 19 |

*Conversion Needed*
*Add binary 0110 (i.e.6)*

$$C = K + Z_8Z_4 + Z_8Z_2$$

# DECIMAL ADDER

- When C = 0
  - No conversion needed.
- When C = 1
  - Add 6 (0110) to get BCD answer.

$$C = K + Z_8Z_4 + Z_8Z_2$$